Hashing Passwords and Sensitive Data

slow, salted, and never reversible

Watch the 4-min video

In one line: store passwords only as slow, salted hashes from a password hashing function (Argon2id, scrypt or bcrypt) tuned to about 100 ms; for data you must get back, like card numbers, use tokenisation or encryption instead.

1 Ten million rows on a forum

A database leaks. Plain-text passwords: every account is gone in minutes, and on other sites too, because people reuse passwords. Plain SHA-256: most are cracked within days. Argon2id with sane parameters: attackers recover only the weakest few. The difference is a few lines of code.

2 Hash, don't store, and why SHA-256 fails

A hash is one-way: easy to compute, infeasible to invert. At login you hash what the user typed and compare. But attackers don't invert hashes; they **guess**: take a list of leaked passwords and dictionary words, hash each, compare. SHA-256 is built to be fast, and one modern GPU computes **billions** of SHA-256 hashes per second. Adding a salt doesn't change that speed. For passwords, fast is bad.

3 Password hashing functions

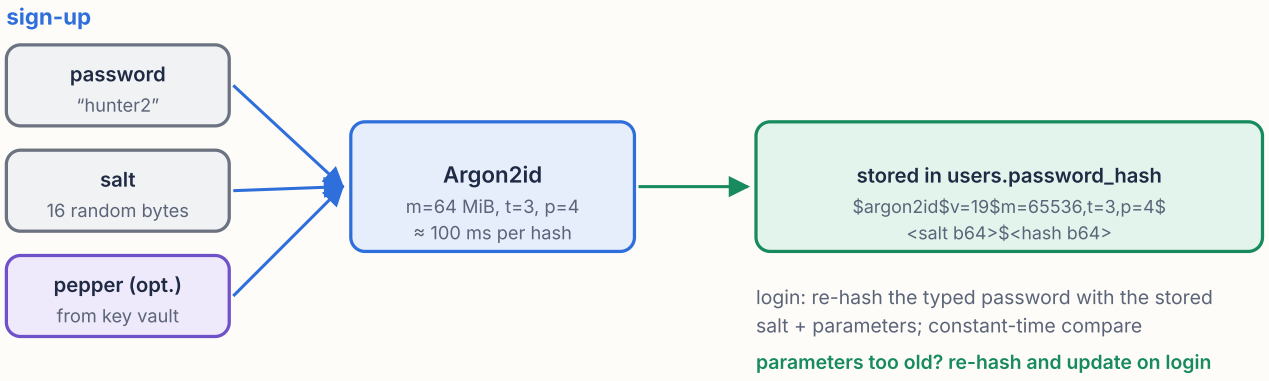


Figure 1. The stored string carries the algorithm, parameters and salt, so you can raise the cost later and upgrade hashes as users log in.

Function	Slows attackers by	OWASP minimum (Password Storage Cheat Sheet)
Argon2id (RFC 9106, recommended)	time + memory (memory-hard)	m = 19 MiB, t = 2, p = 1 (or more memory, fewer passes)
scrypt	time + memory	$N = 2^{17}$, r = 8, p = 1
bcrypt	time (cost factor)	cost 10 or more; input limited to 72 bytes
PBKDF2-HMAC-SHA256	iterations only	600,000 iterations; use when FIPS-140 compliance is required

Each guess should cost about **100 ms** and, for Argon2id and scrypt, tens of MiB of memory, which makes GPU and ASIC attacks expensive. The video's `m=64 MiB, t=3, p=4` is one of RFC 9106's recommended settings. Benchmark on your login servers; raise cost as hardware improves; limit concurrent hashing so a login flood can't exhaust memory.

4 Salt and pepper

A **salt** is random, unique per user (16 bytes is typical) and stored with the hash. Same password, different hashes: rainbow tables are useless and each hash must be attacked on its own. The libraries generate it for you. A **pepper** is one secret shared by all hashes and kept **outside** the database (key vault or HSM), applied as an HMAC; if only the database leaks, the attacker is missing it.

5 Data you need back: tokenise or encrypt

A card number for the next payment can't be hashed. **Tokenisation** keeps the real value in a vault and gives your systems a random token that means nothing elsewhere; restrict and log detokenisation. For values you only need to *look up* (national ID, email), use a keyed HMAC, not a plain hash: low-entropy inputs like 9-digit IDs can be brute-forced in seconds.

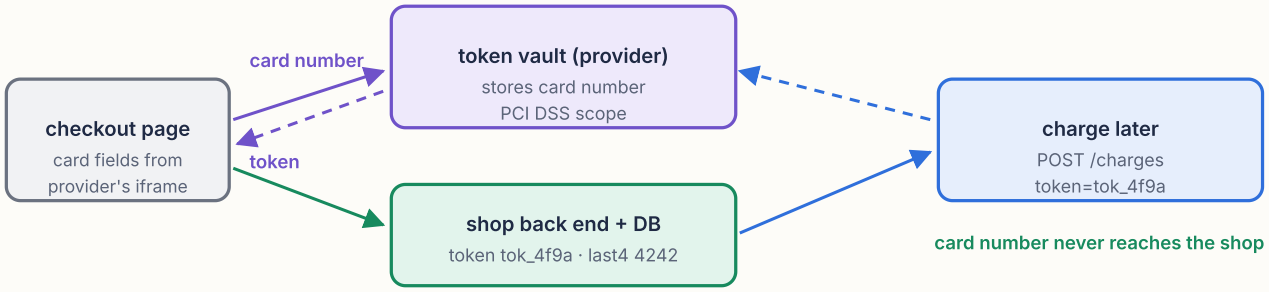


Figure 2. Saved cards via tokenisation: the shop stores a token and the last four digits; the provider's vault holds the card number.

WORKED EXAMPLE: ONE-CLICK CHECKOUT

The checkout page loads card fields from the payment provider (hosted iframe or SDK), so the card number goes straight to the provider's vault. The shop receives `tok_4f9a` and stores only `token, last4=4242, expiry, brand`. To charge again, the back end sends the token. With no card numbers stored, processed or transmitted by its own servers, most of the shop's systems fall outside PCI DSS scope.

COMMON MISTAKES

- MD5, SHA-1 or SHA-256 for passwords, even with a salt.
- Encrypting passwords (reversible) instead of hashing them.
- Writing your own scheme instead of a vetted library (argon2-cffi, bcrypt, libsodium).
- Logging request bodies on the login endpoint, which writes plain passwords to logs.

6 Key terms

Password hashing function: deliberately slow, tunable hash: Argon2id, scrypt, bcrypt

Salt: random per-user value stored with the hash

Pepper: shared secret kept outside the database

Memory-hard: needs lots of RAM per guess, hurting GPUs/ASICs

Rainbow table: precomputed hash → password lookup, defeated by salts

Tokenisation: swap a sensitive value for a meaningless token

7 Interview questions

Q1. How should passwords be stored? BASIC

Hashed with a slow, salted password hashing function, Argon2id (recommended) or bcrypt, with a cost tuned to roughly 100 ms, optionally plus a pepper kept outside the database.

Q2. Why not SHA-256? BASIC

It's designed to be fast: one GPU tests billions of guesses per second, so dictionary attacks against a leaked table succeed quickly.

Q3. You inherit a table of unsalted MD5 password hashes. How do you migrate? SCENARIO

Immediately wrap each: store Argon2id(MD5 hash) with a fresh salt, so no weak hash remains. On each login, verify and replace with a plain Argon2id hash of the password.

Q4. Your app must store card numbers for repeat payments. What do you do? SCENARIO

Don't store them: use the payment provider's tokenisation, keeping only the token and last four digits, which shrinks PCI DSS scope.

Q5. Why is a salted SHA-256 hash of a national ID number still weak? TRICKY

The input space is tiny (about 10^9 values), so the attacker hashes every possible ID with the salt in seconds. Use a keyed HMAC with the key in a vault, or encrypt.

Q6. What is a pepper, and where do you keep it? TRICKY

A secret added to every password hash (usually via HMAC) and stored outside the database, in a key vault or HSM, so a database-only leak can't be cracked offline.

Next: SD-M-012 · OWASP Top 10 backendroadmap.com